

Implementarea unui protocol de comunicare în cadrul FreeBSD

COORDONATORI:
Prof. Dr. Viorel Negru
Asist. Drd. Mario Reja

ABSOLVENT:
Răduică Iulian Robert

TIMIȘOARA

2021

Problema propusă

Pentru ca aplicațiile din userland să își îndeplinească îndatoririle, sunt nevoite să interacționeze cu kernelul pentru a accesa resursele administrate de acesta.

Lucrarea de față studiază comunicarea dintre userland și kernel folosind subsistemul de rețea al sistemului de operare FreeBSD.

În acest sens, am ales implementarea unui protocol de comunicare rudimentar pentru a ușura înțelegerea modului de funcționare a protocoalelor, mai complexe, deja existente precum IP.

Soluții similare

ioctl

Interfața *ioctl* este folosită pentru comunicare între aplicații din userspace și drivere de dispozitiv, bazându-se pe apelul de sistem cu același nume, această interfață a fost introdusă pentru a facilita comunicarea între userspace și perifericele hardware (dispozitive) fără a polua kernelul cu apeluri de sistem.

Netlink

Netlink este un protocol ce facilitează comunicare aplicațiilor de userspace cu kernelul, acesta își are originea în Linux, o implementare parțială a acestuia pe FreeBSD s-a făcut în cadrul OpenVSwitch, acesta folosește interfața socket.

Soluții similare

Procfs și sysfs

Sunt alte două mecanisme de comunicare între procese ce se bazează pe sistemul de fișiere virtual (VFS), spre deosebire de sistemele de fișiere fizice (pe disc) acestea sunt o reprezentare a unor zone din memoria principală.
Interacțiunea dintre userspace și kernelspace folosind vfs prezintă avantajul că pune la dispoziția aplicațiilor din userspace un mod de interacțiune cu kernelul. Acesta este foarte ușor de utilizat, prin intermediul apelurilor de sistem *read* și *write* asupra unui file descriptor.
În cadrul sistemelor de operare din familia BSD echivalentul sysfs este sysctl. Acesta nu folosește VFS ca interfață ci un apel de sistem cu același nume și chiar un executabil ce are scopul de a fi mai ușor de folosit în cadrul script-urilor.

Tehnologii folosite

Tehnologiile folosite în realizarea acestui proiect sunt:

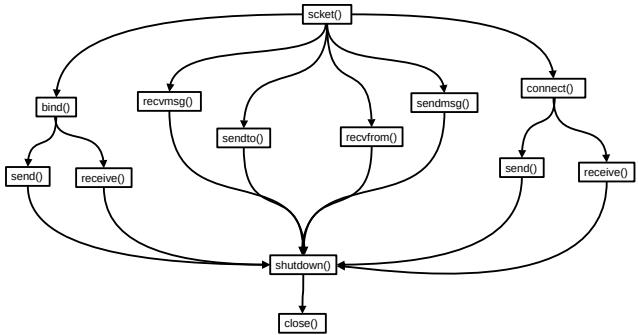
- qemu + KVM
- clang
- Make, PMake
- gdb

Adăugarea de rutine în kernel

- Linkuirea statică a codului direct în kernel;
  - SYSINIT
- Loadable Kernel Module (KLD);
  - DECLARE_MODULE

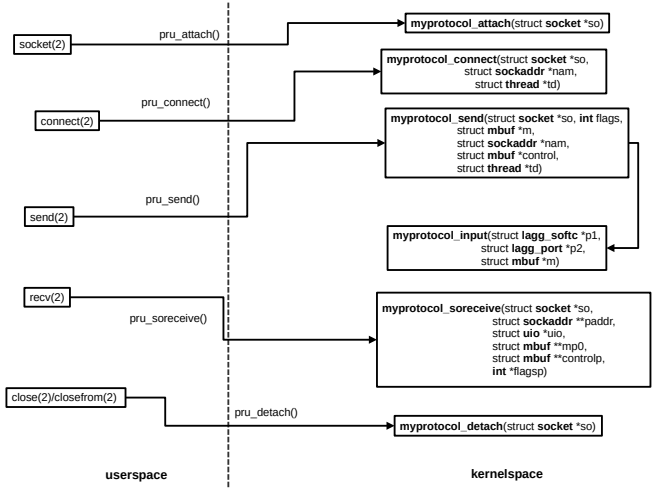
Descriere implementare

Ciclul de viața al unui socket al cărui mod de comunicare este fără conexiune, este ilustrat în următoarea diagramă:



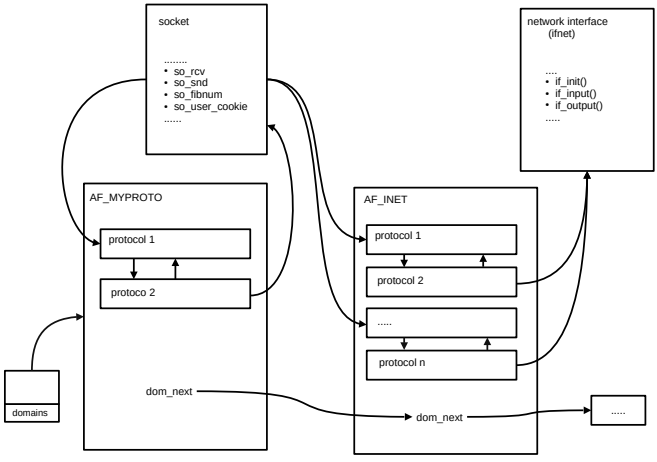
Descriere implementare

Comunicarea între user space și kernel space se va realiza conform următoarei diagrame:



Descriere implementare

Modul în care două protocoale interacționează raportat la interfața socket și la celelalte domenii:



Demo aplicație

Concluzii și direcții viitoare

Obiectivele realizate în cadrul aplicației software asociată acestei lucrări sunt:

- stabilirea unui mediu de dezvoltare adecvat pentru a depana module de kernel;
- înregistrarea unui domeniu de comunicare;
- adăugarea a două protocoale de rețea în cadrul domeniului;
- implementarea unei rutine ce permite eliminarea unui domeniu de comunicare în timpul rulării;
- definirea uni canal de comunicare între funcțiile de librărie din userspace și un protocol de comunicare în rețea;
- stabilirea unui canal de comunicare între două protocoale din cadrul unui domeniu de comunicare;

Concluzii și direcții viitoare

- O primă completare a acestei aplicații este îmbunătățirea depanării de la distanță pentru a permite execuția codului linie cu linie;
- O altă caracteristică care poate fi implementată este comunicarea sincronă, aceasta ar oferi performanțe sporite;
- De asemenea, definirea a mai multor membrii din structura *pr_usrreqs* poate fi considerată o direcție viitoare;